

后端实习生笔试项目：「录音转写服务」API

项目背景

请实现一个简化版的"录音转写与智能摘要"后端服务：客户端上传音频文件，服务端异步完成"转写"（Mock 模拟 或者调用真实的 ASR API）与"智能摘要"（调用真实 LLM API），客户端可查询任务状态与结果。

时间与提交要求

完成周期：收到题目后 4 个自然日内提交（预计实际工作量 12~16 小时，请自行安排）

技术栈：Python / Go / Node.js 任选其一；数据库任选（推荐 PostgreSQL / MySQL / SQLite）

提交内容：

1. GitHub / Gitee 仓库链接（**必须保留完整的 commit 历史**，禁止一次性提交全部代码）
2. `README.md`：运行方式、架构说明（一张简单的架构/流程图）、表结构设计说明、技术取舍、已知问题与未完成项
3. **必须提供服务一键启动方式**（若确有困难，提供详尽的本地运行步骤，但会影响评分）
4. 一份可导入的 API 调试文件（Postman Collection / `.http` 文件 / curl 脚本，任选）

核心功能（必做，P0）

1. 上传录音接口

...

POST /v1/recordings (multipart/form-data, 字段: file)

...

- 接受音频文件上传（仅需校验：文件存在、大小 ≤ 50MB、扩展名为 wav/mp3/m4a/aac 之一）
- 文件保存到本地磁盘（不要求对接对象存储）
- 创建一条录音记录与一个处理任务，**立即返回**，不得同步等待处理完成：

```json

{ "recording\_id": "...", "task\_id": "...", "status": "pending" }

```

2. 异步处理流水线

任务状态机：`pending` → `transcribing` → `summarizing` → `done`，任一环节可失败进入 `failed`

转写阶段 (Mock): 不要求接入真实 ASR。要求模拟真实行为: 随机耗时 5~15 秒, 约 20% 概率失败; 成功后产出一段固定或随机的 transcript 文本 (自行生成)
摘要阶段 (真实 LLM): 调用真实的 LLM API, 对 transcript 生成结构化结果:

```
```json
{ "summary": "一句话摘要", "key_points": ["要点 1", "要点 2"], "todos": ["待办 1"]}
```
```

- LLM 可使用任意免费/低成本渠道 (如 DeepSeek、智谱、Groq 免费额度, 或本地 Ollama)。若确实无法获取任何 API Key, 可 Mock 该步骤, 但需在 README 中说明, 且此项得分降低

- 必须处理: LLM 调用超时、返回内容不符合预期格式 (如无法解析出 JSON) 的情况
- 异步实现方式不限: 语言自带并发原语 (goroutine / asyncio / worker threads)、Redis 队列、Celery 等均可, 但需在 README 中说明选型理由, 以及**你的方案在服务重启时未完成的任务会怎样**

3. 任务与结果查询

```
```
GET /v1/tasks/{task_id} 查询任务状态 (processing 中需能体现当前阶段)
GET /v1/recordings?page=&page_size= 录音列表, 分页, 按创建时间倒序, 含每条的最新任务状态
GET /v1/recordings/{id} 录音详情, 若处理完成则包含 transcript 与摘要结果
POST /v1/tasks/{task_id}/retry 失败任务重试 (仅 failed 状态可重试, 需处理重复请求)
DELETE /v1/recordings/{id} 删除录音 (含文件与关联数据)
```
```

4. 工程要求

- 统一的错误响应结构与合理的 HTTP 状态码 (404 / 400 / 409 等场景需正确区分)
- 关键路径有日志 (能通过日志还原一个任务的完整生命周期)
- 数据库表结构通过迁移脚本或 SQL 文件提供

加分项 (选做, 完成任意一项即显著加分, 请勿为了加分牺牲核心功能质量)

1. 失败自动重试: 转写/摘要失败后自动重试 (最多 3 次, 指数退避), 重试耗尽才置为 `failed`
2. 服务重启恢复: 服务重启后, 处理中/排队中的任务能被恢复继续执行, 而非永久卡死
3. LLM 流式输出: 提供 `GET /v1/recordings/{id}/summary/stream`, 以 SSE 方式流式返回摘要生成过程

4. 上传幂等：客户端可传入幂等键（或基于文件哈希去重），重复上传同一文件不产生重复任务
5. 并发控制：限制同时处理的任务数（如最多 3 个并发），其余排队
6. 测试：为状态机流转 / 核心接口编写单元测试或集成测试
7. 简单部署：将服务部署到任意公网环境（免费云 / 学生机均可），提供可访问地址

明确不考察的内容（请不要花时间）

- 用户注册 / 登录 / 鉴权
- 前端页面
- 高并发性能优化（正确性优先于性能）

关于 AI 工具的使用

- 允许使用 Copilot / Claude / ChatGPT 等任何 AI 工具辅助开发
- 面试时会有代码答辩环节：你需要能解释任意一段代码的设计意图，如果无法解释自己提交的代码，会影响面试结果